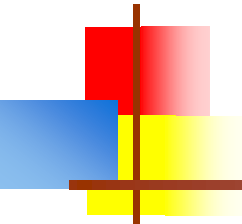


CSS

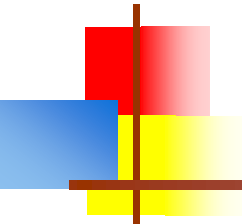
Applications to HTML





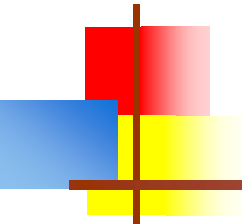
The problem with HTML

- HTML was originally intended to describe the *content* of a document
- Page authors didn't have to describe the layout--the browser would take care of that
- This is a good engineering approach, but it didn't satisfy advertisers and “artists”
 - Even people that actually had something to say wanted more control over the appearance of their web pages
- As a result, HTML acquired more and more tags to control *appearance*
 - Content and appearance became more intertwined
 - Different browsers displayed things differently, which is a real problem when appearance is important



Cascading Style Sheets

- A Cascading Style Sheet (CSS) describes the appearance of an HTML page in a separate document
- CSS has the following advantages:
 - It lets you separate content from presentation
 - It lets you define the appearance and layout of all the pages in your web site in a single place
 - It can be used for HTML, XHTML, and XML pages
- CSS lets you style the elements on your web page like colored text, position, and size.
- CSS works in conjunction with HTML, but is not HTML itself.

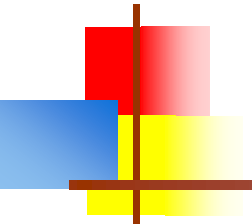


The CSS Rule



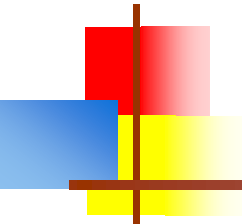
`selector { property: value; }`

- A block of CSS code is a rule.
- The rule starts with a selector.
- It has sets of properties and values.
- A property-value pair is a declaration.



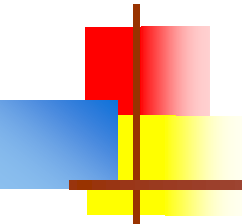
Connecting CSS to HTML

- 3 ways
 - "Inline"
 - "Embedded"
 - "External"



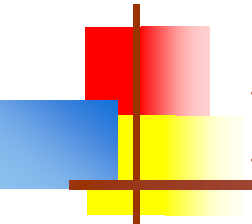
Using style sheets

- There are three ways of using CSS:
 - **External style sheet**
 - This is the most powerful
 - Applies to both XHTML/HTML and XML
 - All of CSS can be used
 - **Embedded style sheet**
 - Applies to XHTML/HTML, *not* to XML
 - All of CSS can be used
 - **Inline styles**
 - Applies to XHTML/HTML, not to XML
 - Limited form of CSS syntax



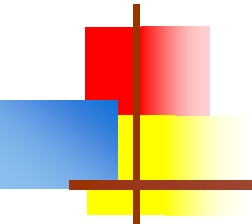
External style sheets

- In XHTML/HTML, within the `<head>` element:
`<link rel="stylesheet" type="text/css"
href="Style Sheet URL">`
- Note: "text/css" is the MIME type
- Shared resource for several pages.
- Reduced file size & bandwidth
- Easy to maintain in larger projects.
- Preferred way to use CSS!



Embedded style sheets

- In HTML, within the `<head>` element:
`<style type="text/css">`
 `<!--`
 CSS Style Sheet
 `-->`
`</style>`
- Note: Embedding the style sheet within a comment is a sneaky way of hiding it from older browsers that don't understand CSS
 - There aren't many of these around any more
- Uses `<style>` tag.
 - Can only be used in one html file



Inline style sheets

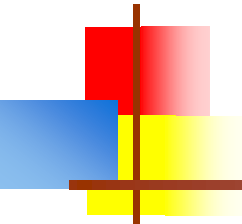
- The **style** attribute can be added to any HTML element:

`<html-tag style="property: value">` or

`<html-tag style="property: value;
property: value; ...; property: value">`

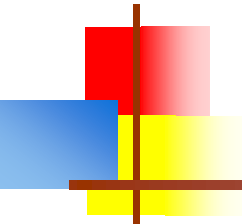
`<p style="color:red">Some text.</p>`

- Advantage:
 - Useful if you only want a small amount of markup
- Disadvantages:
 - Mixes display information into HTML
 - Clutters up HTML code (**NOT Preferred**)
 - Can't use full range of CSS features



CSS syntax, I

- CSS syntax is very simple--it's just a file containing a list of **selectors** (to choose tags) and **descriptors** (to tell what to do with them):
 - Example: `h1 {color: green; font-family: Verdana}` says that everything included in `h1` (HTML heading level 1) tags should be in the Verdana font and colored green
- A CSS file is just a list of these selector/descriptor pairs
 - Selectors may be simple HTML tags or XML tags, but CSS also defines some ways to combine tags
 - Descriptors are defined in CSS itself, and there is quite a long list of them



CSS syntax

- The general syntax is:

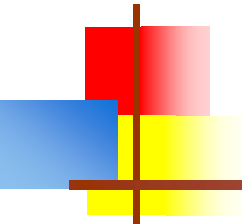
selector { property: value; }

- or

*selector, ..., selector {
property: value;
...
property: value;
}*

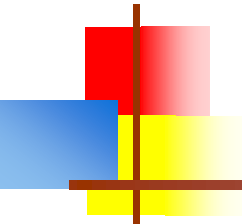
- where

- *selector* is the tag to be affected (the selector is case-sensitive if and only if the document language is case-sensitive)
- *property* and *value* describe the appearance of that tag
- Spaces after colons and semicolons are optional
- A semicolon must be used *between* **property:value** pairs; a semicolon after the last pair is recommended but optional



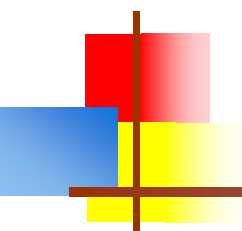
Example of CSS

- `/* This is a comment */`
- `h1,h2,h3 {font-family: Arial, sans-serif;} /* use 1st available font */`
- `p, table, li, address {` `/* apply to all these tags */`
 `font-family: "Courier New";` `/* quote values containing spaces */`
 `margin-left: 15pt;` `/* specify indentation */`
 `}`
- `p, li, th, td {font-size: 80%;}` `/* 80% of size in containing element */`
- `th {background-color:#FAEBD7}` `/* colors can be specified in hex */`
- `body { background-color: #ffffff;}`
- `h1,h2,h3,hr {color:saddlebrown;}` `/* adds to what we said before */`
- `a:link {color:darkred}` `/* an unvisited link */`
- `a:visited {color:darkred}` `/* a link that has been visited */`
- `a:active {color:red}` `/* a link now being visited */`
- `a:hover {color:red}` `/* when the mouse hovers over it */`

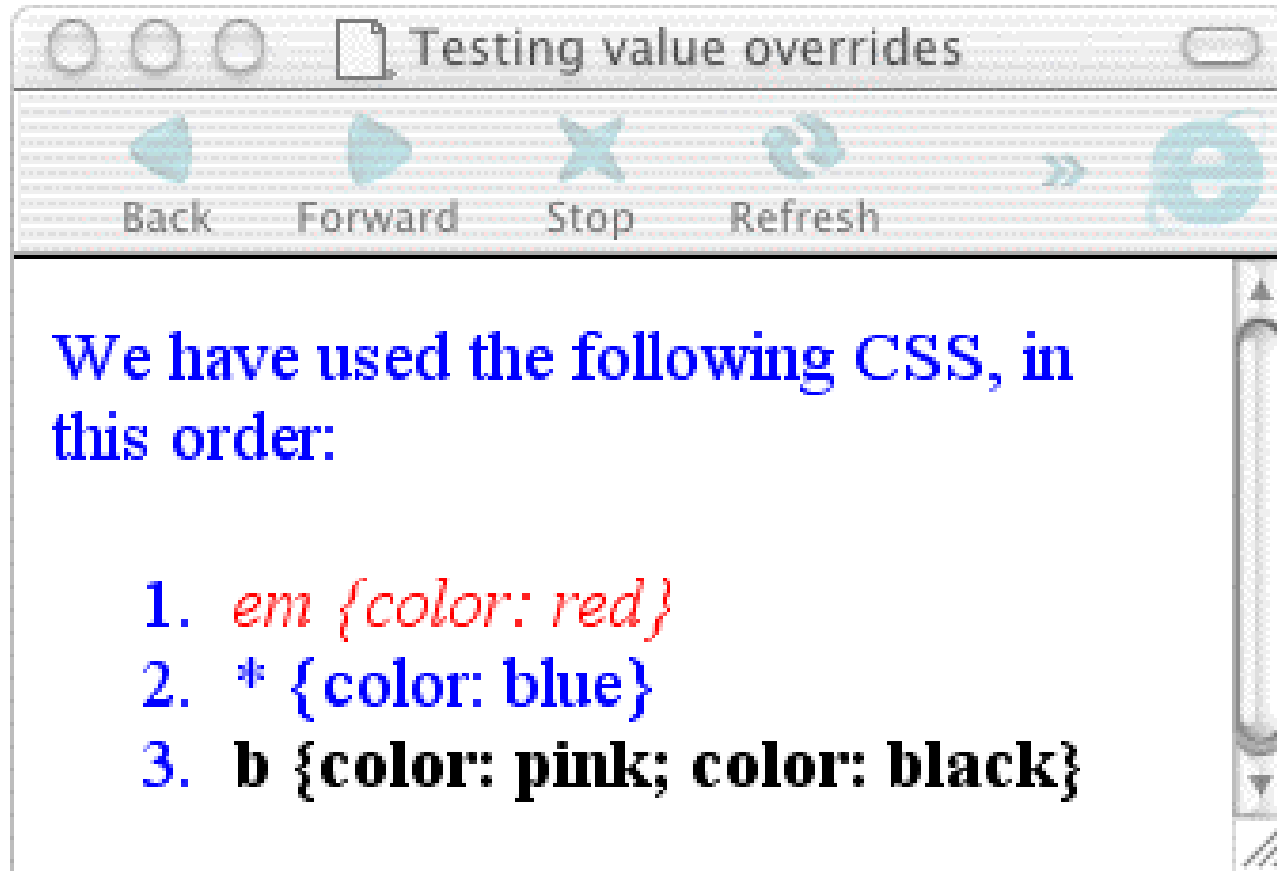


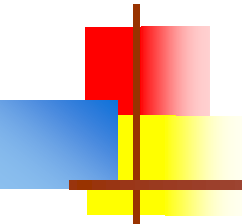
More about selectors, I

- As we have seen, an XML or HTML tag can be used as a **simple element selector**:
`body { background-color: #ffffff }`
- You can use multiple selectors:
`em, i {color: red}`
You can repeat selectors:
`h1, h2, h3 {font-family: Verdana; color: red}`
`h1, h3 {font-weight: bold; color: pink}`
 - When values disagree, the last one overrides any earlier ones
- The **universal selector** `*` applies to any and all elements:
`* {color: blue}`
 - When values disagree, more specific selectors override general ones (so `em` elements would still be red)



Example of overriding





More about selectors, II

A **descendent selector** chooses a tag with a specific ancestor:

- `p code { color: brown }`
 - selects a `code` if it is somewhere inside a paragraph

■ A **child selector** `>` chooses a tag with a specific parent:

`h3 > em { font-weight: bold }`

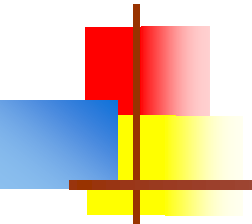
selects an `em` only if its immediate parent is `h3`

■ An **adjacent selector** chooses an element that immediately follows another:

`b + i { font-size: 8pt }`

Example: `<b>I'm bold and</b> <i>I'm italic</i>`

Result will look something like: **I'm bold and** *I'm italic*



Selector: Position

```
p em { color: yellow; }
```

- Selects all **em** elements that are within a paragraph.

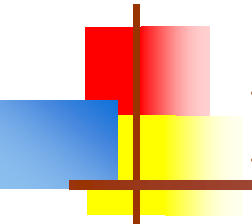
```
<p>This is <em>important.</em></p>
```

- Position selectors are more specific
- They look for elements *inside* other elements
- We separate nested elements with a space

```
ul li a strong { color: purple; }
```

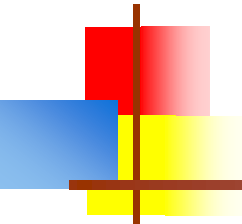
- Means "find a strong tag inside a link inside a list item in an unordered list"

```
<ul> <li> <a href="programs.html">Our  
<strong>program</strong></a> </li> </ul>
```

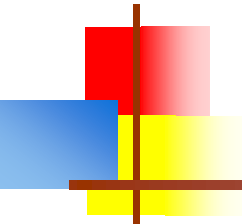
More about selectors, III

- A **simple attribute selector** allows you to choose elements that have a given attribute, regardless of its value:
 - Syntax: *element*[*attribute*] { ... }
 - Example: `table[border] { ... }`
- An **attribute value selector** allows you to choose elements that have a given attribute with a given value:
 - Syntax: *element*[*attribute*="*value*"] { ... }
 - Example: `table[border="0"] { ... }`



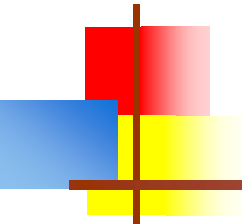
More about values

- As we have seen, the syntax for a CSS rule is:
selector, ..., selector { property: value; ... property: value }
- The value is whatever occurs between the colon and the semicolon (or closing brace)
- Example: ** {font-family: Trebuchet, Verdana, sans-serif;}*
 - This means to use the Trebuchet font for everything, if it is available; else use the Verdana font, if available; else use whatever sans serif font the browser uses as default
- *section {border: thin solid blue;}*
 - This means to put a borders around *section* elements; the borders are to be thin *and* solid *and* blue



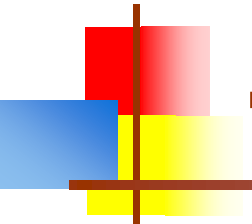
Reusing Code

- As a general coding principle, **Don't Repeat Yourself**.
- To reuse CSS, we use IDs and classes.
- **ID** -- Should only apply to one element on a webpage, i.e., a webpage only has one footer.
 - The "#" is how you tell CSS "this is an id."
- **Class** -- Lots of elements can have the same class, i.e., There can be many warnings on one webpage.
 - The "." is how you tell CSS "this is a class name."



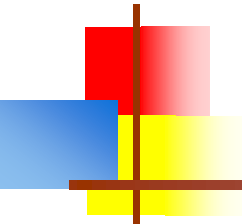
The class attribute

- The **class** attribute allows you to have different styles for the same element
 - In the style sheet:
`p.important {font-size: 24pt; color: red}`
`p.fineprint {font-size: 8pt}`
 - In the HTML:
`<p class="important">The end is nigh!</p>`
`<p class="fineprint">Offer ends 1/1/97.</p>`
- To define a selector that applies to any element with that class, just omit the tag name (but keep the **dot**):
`.fineprint {font-size: 8pt}`



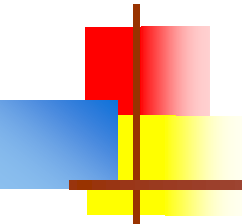
The id attribute

- The **id** attribute is defined like the **class** attribute, but uses **#** instead of **.**
 - In the style sheet:
`p#important {font-style: italic}` or `# important {font-style: italic}`
 - In the HTML:
`<p id="important">`
- **class** and **id** can both be used, and do not need to have different names:
`<p class="navigation_bar" id="navigation_bar">`
- **Important difference:** **id** is used to specify a unique identifier, so it should only be used once in any given document



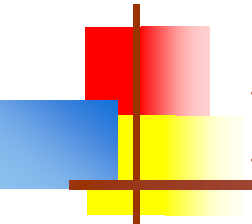
div and span

- **div** and **span** are HTML elements whose only purpose is to hold CSS information
- **div** ensures there is a line break before and after (so it's like a paragraph); **span** does not
- Example:
 - CSS: `div {background-color: #66FFFF}`
`span.color {color: red}`
 - HTML: `<div>This div is treated like a paragraph, but <span class="color">this span</span> is not.</div>`



Cascading order

- Styles will be applied to HTML in the following order:
 1. Browser default
 2. External style sheet
 3. Internal style sheet (inside the `<head>` tag)
 4. Inline style (inside other elements, outermost first)
- When styles conflict, the “nearest” (most recently applied) style wins



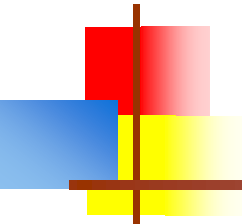
Example of cascading order

- External style sheet:

```
h3 { color: red;
      text-align: left;
      font-size: 8pt
    }
```
- Internal style sheet:

```
h3 { text-align: right;
      font-size: 20pt
    }
```
- Resultant attributes:

```
color: red;
text-align: right;
font-size: 20pt
```

Cascading

- Styles "cascade" down until changed

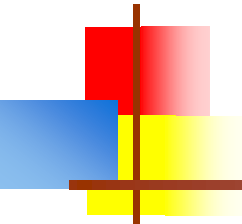
```
p { color:blue; font-family: 'Helvetica'; }
```

```
.red { color: red; }
```

```
#special { font-family: Arial; }
```

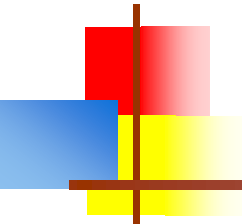
In HTML:

```
<p>Paragraph</p> <p class="red">Paragraph</p> <p  
class="red" id="special">Paragraph</p>
```



Cascading Priority

- Your browser assigns different priorities to CSS depending on the type of selector.
 1. Important! - **Most Important**
 2. In line CSS
 3. ID
 4. Class
 5. Element - **Least Important**
- Your browser also assigns priority based on the specificity of the selection. More specific selectors have higher priority.



Cascading Priority

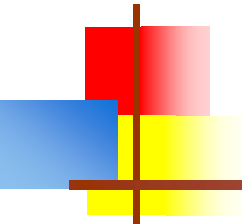
- The tie-breaker is position. Rules lower in the file overwrite rules higher in the file

```
a { background-color: yellow; }
```

```
a { background-color: teal; }
```

```
// This rule wins
```

```
a { background-color: black; }
```



A novel example: XHTML

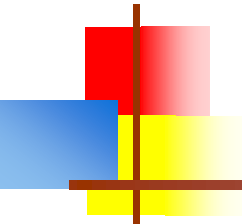
- ```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head>
 <title>The Great American Novel</title>
 <link rel="stylesheet" href="novel.css" type="text/css" />
</head>

<body>
 <p class="foreword">This is the great American novel.</p>

 <div class="chapter">
 <p>It was a dark and stormy night.</p>

 <p>Suddenly, a shot rang out!</p>
 </div>
</body>
</html>
```



# A novel example: CSS

---

```
.chapter {
 font-family: Papyrus,
 "Comic Sans MS", fantasy;
}
```

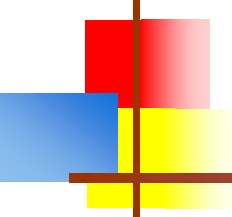
```
.chapter {
 display: block
}
```

```
.chapter:before {
 content: "New chapter: "
}
```

```
.chapter:first-letter {
 font-size: 200%;
 float: left
}
```

```
p {
 display: block
}
```

```
p.foreword {
 border: solid red;
 padding: 10px;
 font-family: Impact;
 color: blue;
}
```



# A novel example: Results

Opera (Macintosh)

**This is the great American novel.**

New chapter:

It was a dark and stormy night.

Suddenly, a shot rang out!

Firefox (Macintosh)

**This is the great American novel.**

New chapter:

It was a dark and stormy night.

Suddenly, a shot rang out!

IE 6 (Windows)

**This is the great American novel.**

It was a dark and stormy night.

Suddenly, a shot rang out!

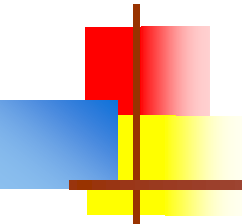
IE 8 beta (Windows)

**This is the great American novel.**

New chapter:

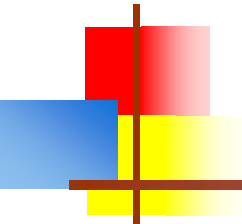
It was a dark and stormy night.

Suddenly, a shot rang out!



# Some font properties and values

- **font-family:**
  - inherit (same as parent)
  - Verdana, "Courier New", ... (if the font is on the client computer)
  - serif | sans-serif | cursive | fantasy | monospace  
(Generic: your browser decides which font to use)
- **font-size:**
  - inherit | smaller | larger | xx-small | x-small | small | medium | large | x-large | xx-large | 12pt
- **font-weight:**
  - normal | bold | bolder | lighter | 100 | 200 | ... | 700
- **font-style:**
  - normal | italic | oblique



# Shorthand properties

---

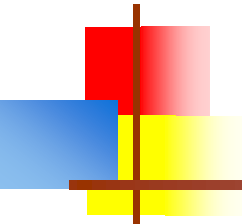
- Often, many properties can be combined:

```
h2 { font-weight: bold; font-variant: small-caps; font-size: 12pt; line-height: 14pt; font-family: sans-serif }
```

can be written as:

```
h2 { font: bold small-caps 12pt/14pt sans-serif }
```

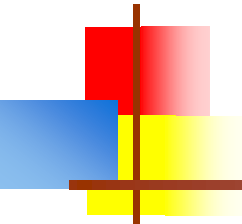




# CSS Color Values

---

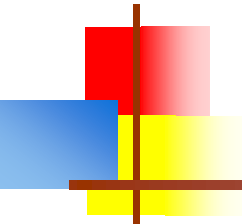
- Your browser can accept colors in many different ways:
- Color name (ex. red)
- Hexadecimal value (ex. #FF0000)
- RGB value (ex. rgb(255, 0, 0))
- HSL value (ex. hsl(0, 100%, 100%))
- The 17 standard colors are:  
aqua, black, blue, fuchsia, gray, grey, green, lime, maroon, navy, olive, purple, red, silver, teal, and yellow.



# Colors and lengths

---

- color: and background-color:
  - aqua | black | blue | fuchsia | gray | green | lime | maroon | navy | olive | purple | red | silver | teal | white | #FF0000 | #F00 | rgb(255, 0, 0) | *Additional browser-specific names (not recommended)*
- These are used in measurements:
  - em, ex, px, %
    - font size, x-height, pixels, percent of inherited size
  - in, cm, mm, pt, pc
    - inches, centimeters, millimeters, points (1/72 of an inch), picas (1 pica = 12 points), relative to the inherited value



# Property Values

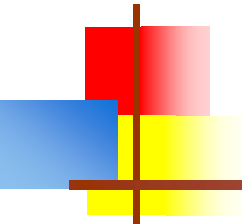
---

- Each property can have one or more comma separated values.

```
p {
 color: white;
 background-color: red;
 font-family: Arial, sans-serif;
}
```

```
p { font-family: "Times New Roman"; font-family: serif;
font-family: "Arial", sans-serif; }
```

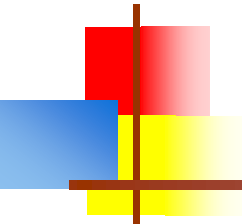
- Specific font name
- Generic name
- Comma-separated list



# Some text properties and values

---

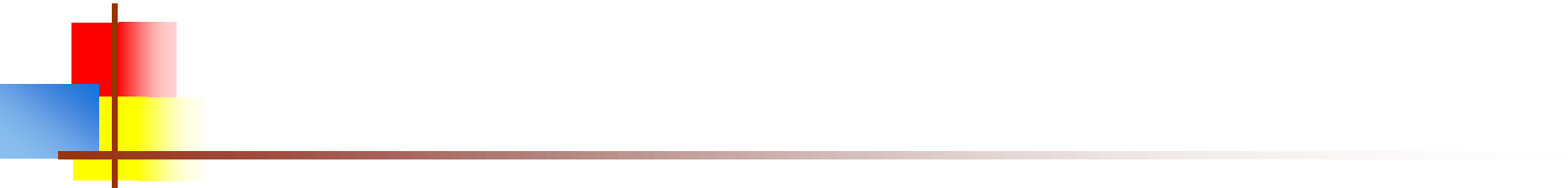
- text-align:
  - left | right | center | justify
- text-decoration:
  - none | underline | overline | line-through
- text-transform:
  - none | capitalize | uppercase | lowercase
- text-indent
  - *length* | *10%* (indents the first line of text)
- white-space:
  - normal | pre | nowrap



# Pseudo-classes

---

- **Pseudo-classes** are elements whose state (and appearance) may change over time
- Syntax: *element:pseudo-class {...}*
  - **:link**
    - a link which has not been visited
  - **:visited**
    - a link which has been visited
  - **:active**
    - a link which is currently being clicked
  - **:hover**
    - a link which the mouse is over (but not clicked)
- Pseudo-classes are allowed anywhere in CSS selectors
- Changing the format of a link when you hover over it is accomplished by using pseudo-classes.
- CSS pseudo-classes are used to add special effects to some selectors.

- 
- Syntax: selector:pseudo-class { property:value; }

Example:

```
a:link { text-de
```

```
a:link { color:#FF0000; } /* unvisited link */
```

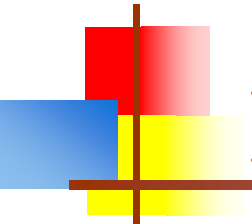
```
a:visited { color:#00FF00; } /* visited link */
```

```
a:hover, a:focus { color:#FF00FF; } /* mouse over or select
with keyboard */
```

```
a:active { color:#0000FF; } /* selected link */
```

a:hover MUST come after a:link and a:visited in the CSS definition to be effective!

a:active MUST come after a:hover in the CSS definition to be effective!



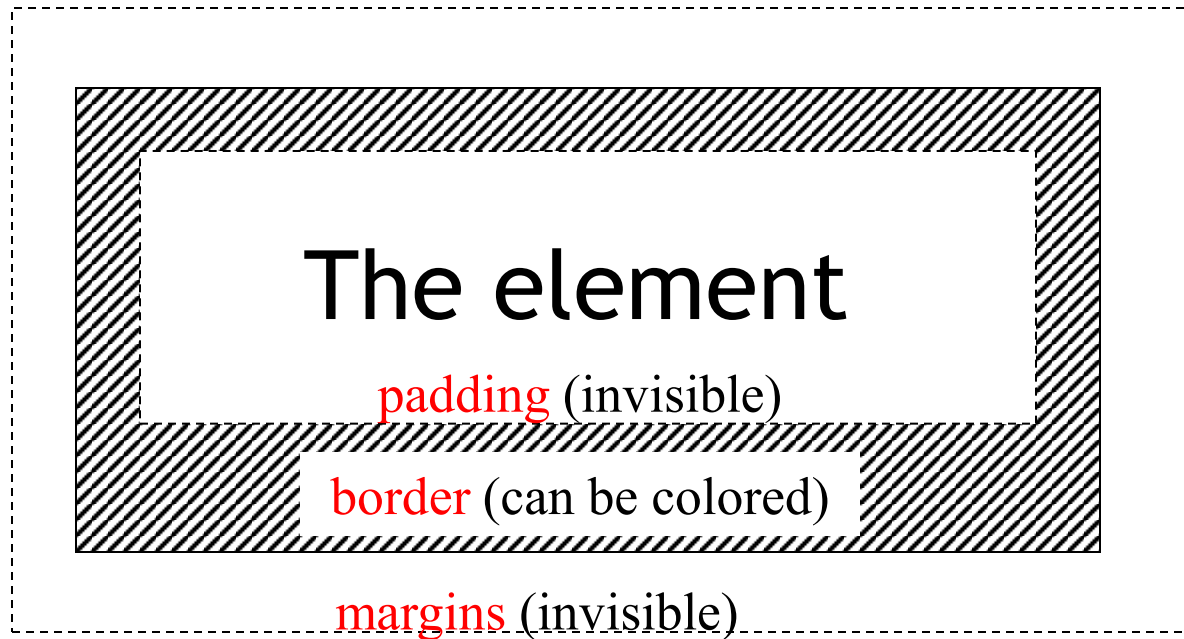
# Layout properties in CSS

---

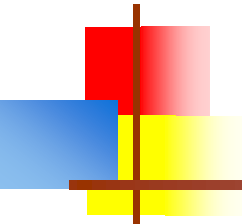
- Style properties that can be defined using CSS can be considered to be grouped into two categories—
  - Layout properties
  - Formatting properties.
- CSS considers each element as a box.
- Box model, made up of the Content area, along with Padding, Border and Margin.
- All Block elements and inline elements are treated as per the box model.

# Boxes

- The invisible box containing a styled element has three special areas:



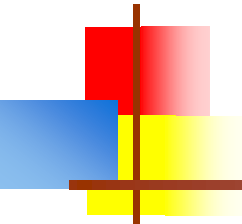




# Padding

---

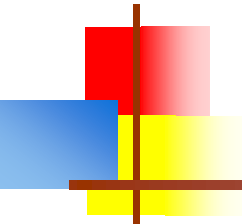
- **Padding** is the extra space around an element, but inside the border
- To set the padding, use any or all of:
  - padding-top: *size*
  - padding-bottom: *size*
  - padding-left: *size*
  - padding-right: *size*
  - padding: *size* to set all four sides at once
- *size* is given in the units described earlier
- Example: sidebar {padding: 1em; padding-bottom: 5mm}



# Borders

---

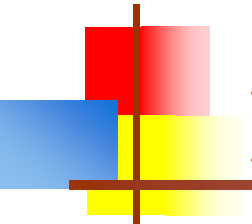
- You can set border attributes with any or all of: `border-top:`, `border-bottom:`, `border-left:`, `border-right:`, and `border:` (all at once)
- The attributes are:
  - The *thickness* of the border: `thin`, `medium` (default), `thick`, or a specific size (like `3px`)
  - The *style* of the border: `none`, `dotted`, `dashed`, `solid`, `double`, `groove`, `ridge`, `inset`, or `outset`
  - The *color* of the border: one of the 16 predefined color names, or a hex value with `#rrggbb` or `rgb(r, g, b)` or `rgb(r%, g%, b%)`
- Example: `section {border-top: thin solid blue;}`
- Note: the special styles (such as `groove`) are not as cool as they sound



# Margins

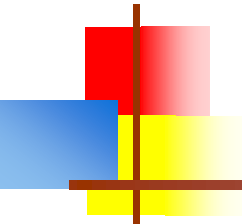
---

- **Margins** are the extra space outside the border
- Setting margins is analogous to setting padding
- To set the margins, use any or all of:
  - `margin-top: size`
  - `margin-bottom: size`
  - `margin-left: size`
  - `margin-right: size`
  - `margin: size` to set all four sides at once



# Box and display interactions

- With `display:none`, contents are invisible and margin, border, and padding settings have no effect
- With `display:block`, margin, border, and padding settings work about as you would expect
- With `display:inline` (which is the default if you don't specify otherwise):
  - Margin, border, and padding settings for `left` and `right` work about as you would expect
  - Margin, border, and padding settings for `top` and `bottom` *do not add extra space above and below the line containing the element*
    - This means that inline boxes will overlap other text



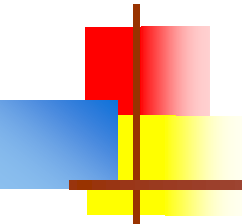
# Sizing elements

---

- Every element has a *size* and a *natural position* in which it would be displayed
- You can set the height and width of **display:block** elements with:
  - **height: *size***
  - **width: *size***
- You *cannot* set the height and width of inline elements (but you can set left and right margins)
- In HTML, you can set the height and width of images and tables (**img** and **table** tags)

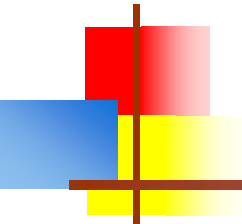
# Setting absolute position

- To move an element to an absolute position on the page
  - `position: absolute` (this is required!) *and also* one or more of
    - `left: size` or `right: size`
    - `top: size` or `bottom: size`
- Confusing stuff:
  - `size` can be positive or negative
  - `top: size` puts an element's top `size` units from the page top
  - `bottom: size` puts an element's bottom `size` units from the page bottom
  - `left: size` puts an element's left side `size` units from the left edge of the page
  - `right: size` puts an element's right side `size` units from the right edge of the page
  - Changing an element's absolute position removes it from the natural flow, so other elements fill in the gap
  - You need to be careful not to overlap other elements



# Setting relative position

- To move an element relative to its natural position, use
  - `position: relative` (this is required!) *and also* one or more of
    - `left: size` or `right: size`
    - `top: size` or `bottom: size`
- Confusing stuff:
  - `size` can be positive or negative
  - to move *left*, make `left` negative or `right` positive
  - to move *right*, make `right` negative or `left` positive
  - to move *up*, make `top` negative or `bottom` positive
  - to move *down*, make `bottom` negative or `top` positive
  - Setting an element's position *does not affect* the position of other elements, so
    - There will be a gap in the element's original, natural position
    - Unless you are very careful, your element will overlap other elements



# Float

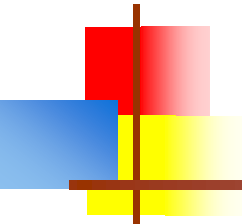
---

- The **float** property pushes an element to the left or right of its container, allowing text and inline elements to wrap around it. It is often used for aligning images within text or creating simple horizontal layouts.

Values: left, right, none (default), or inherit.

Behavior: Floated elements are removed from the normal document flow but still remain part of it. Note that absolute or fixed-position elements ignore this property.

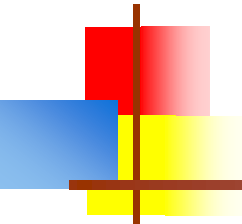




# Clear

---

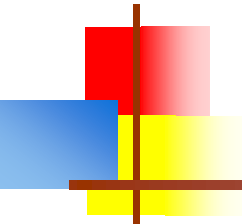
- The **clear** property is the companion to **float**.
- It specifies whether an element should be placed next to a floated element or be pushed below it to start on a new line.
- Values:
  - left: Pushes the element below any left-floated elements.
  - right: Pushes the element below any right-floated elements.
  - both: Pushes the element below both left and right-floated elements.



# Z-Index

---

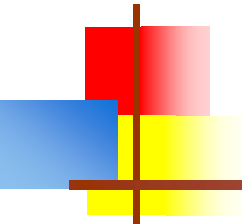
- The **z-index** property controls the stacking order of overlapping elements along the imaginary z-axis (perpendicular to the screen).
- Requirements: It only works on positioned elements (those with position set to **relative**, **absolute**, **fixed**, or **sticky**) and flex/grid items.
- **How it Works:** Higher numerical values are stacked in front of elements with lower values. Negative values can be used to push an element behind others.
- **Stacking Context:** If an element is inside a parent with its own z-index, it belongs to a "stacking context." Its stacking order is relative to its siblings within that context; it cannot move in front of elements in a higher-priority context even with a massive z-index value.



# Choosing good names

---

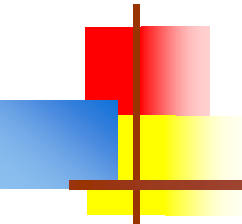
- CSS is designed to *separate content from style*
  - Therefore, names that will be used in HTML or (especially) in XML should describe *content*, *not style*
- Example:
  - Suppose you define `span.huge {font-size: 36pt}` and you use `<span class="huge">` throughout a large number of documents
  - Now you discover your users hate this, so you change the CSS to be `span.huge {font-color: red}`
  - Your name is inappropriate; do you change all your documents?
  - If you had started with `span.important {font-size: 36pt}`, your documents wouldn't look so dumb



# References

---

- Some of the examples in this presentation were taken from the **W3Schools** online tutorial at [http://www.w3schools.com/css/css\\_syntax.asp](http://www.w3schools.com/css/css_syntax.asp)
- Dave Raggett's **Adding a Touch of Style** is a very nice online tutorial at <http://www.w3.org/MarkUp/Guide/Style>
- **Index DOT Css** has also been a great source of information about CSS:  
<http://www.blooberry.com/indexdot/css/index.html>
  - In particular, there is a list of when CSS features were first supported by which browsers (-- means “not yet supported”) at <http://www.blooberry.com/indexdot/css/supportkey/syntax.htm>



# The End

---